

Diseño de Sistemas de Software

INGENIERÍA DE SISTEMAS
ANDRÉS DÍAZ PACE - 2024

Presentaciones



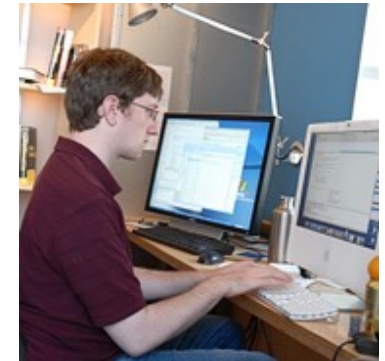
J. Andrés Díaz Pace

andres.diazpace@isistan.unicen.edu.ar



Alfredo Teyseyre

alfredo.teyseyre@isistan.unicen.edu.ar



Modalidad de Dictado

- 2 clases presenciales semanales (lunes y martes)
... eventualmente alguna clase sincrónica virtual (Meet)



- Material adicional
 - Slides, casos de estudio, artículos, enlaces, etc.
- Herramientas
 - Campus Virtual (Moodle)
 - IDE (Eclipse, IntelliJ) para Java, VS (Python)?
 - Editor de diagramas (Visio, etc.)



- Entrega de ejercicios prácticos seleccionados
- Quizzes (vía Moodle)
- Extensión de alguno de los TPs como TP Especial?
- **Parcial**



Contenidos de la Materia

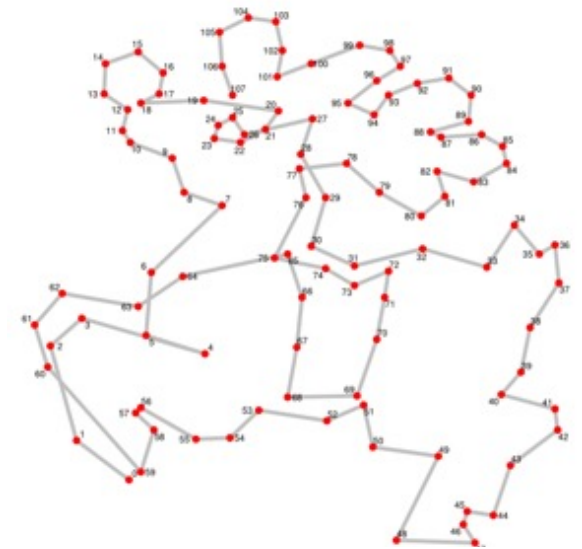
Diseño de software desde una perspectiva arquitectural

- Atributos de calidad
- Patrones y frameworks
- Cliente-servidor, Bases de datos, Microservicios
- Otros estilos y tácticas de arquitectura
 - Introducción a Cloud computing
- Métodos de arquitectura
 - Diseño
 - Documentación
 - Evaluación
- Arquitectura & AI/LLMs?

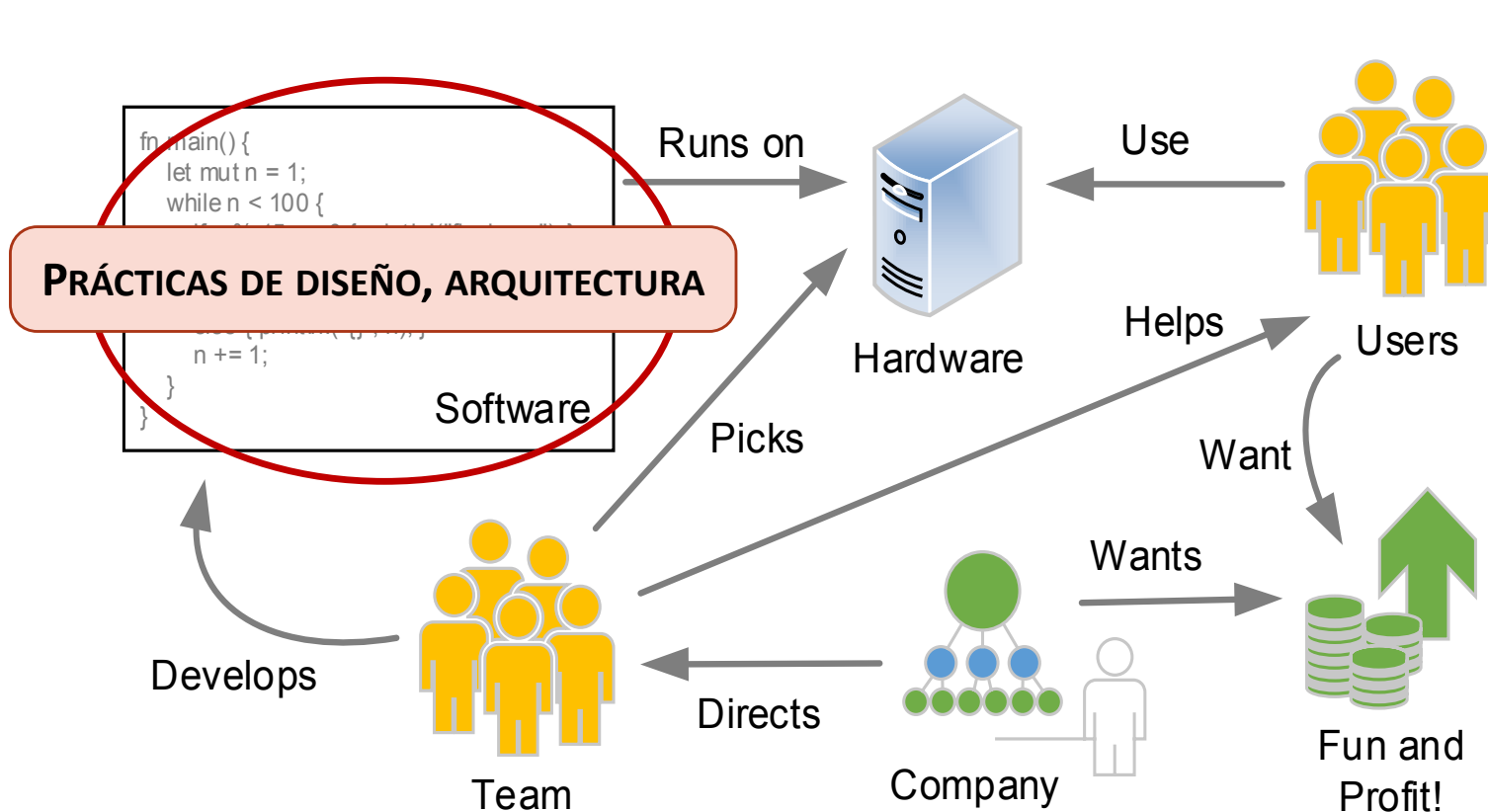


Relación con otras Materias

- Programación Orientada a Objetos
- Bases de datos
- Sistemas operativos, redes, infraestructura
- Metodologías de Desarrollo de software
- ...

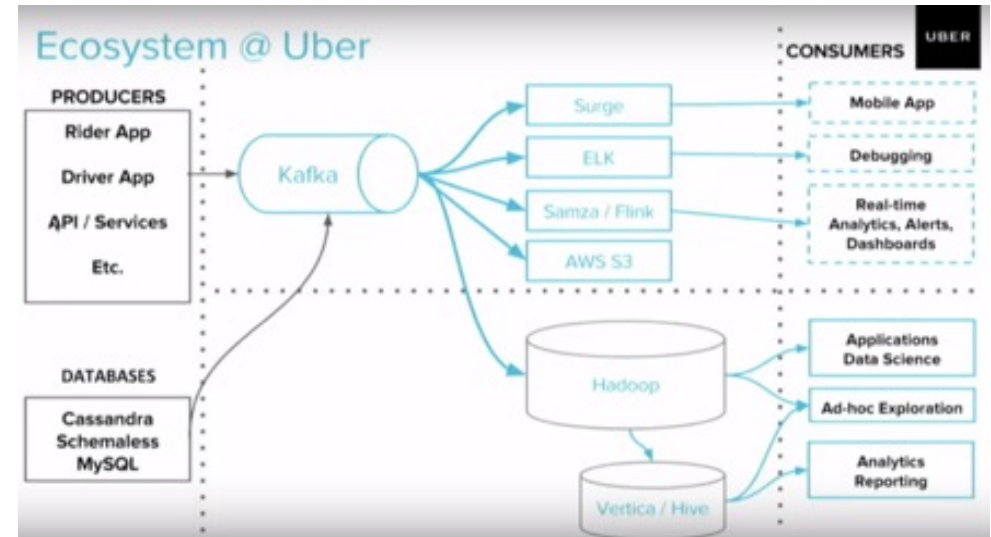


Software en el contexto de un sistema



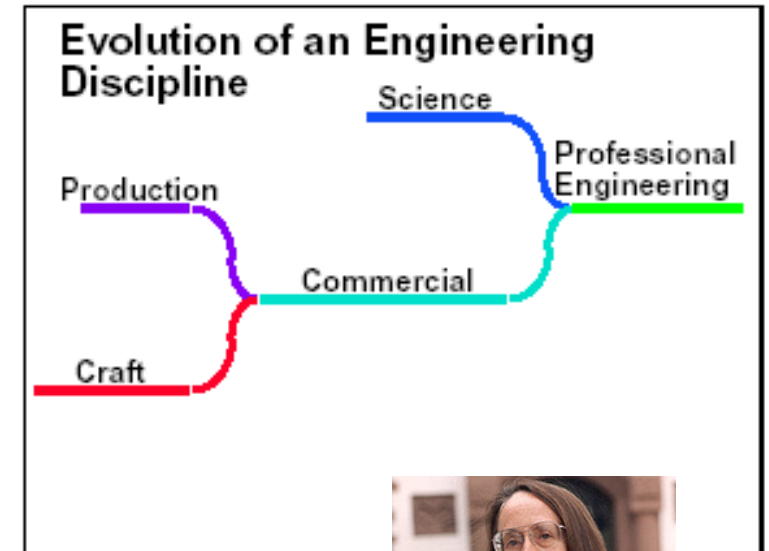
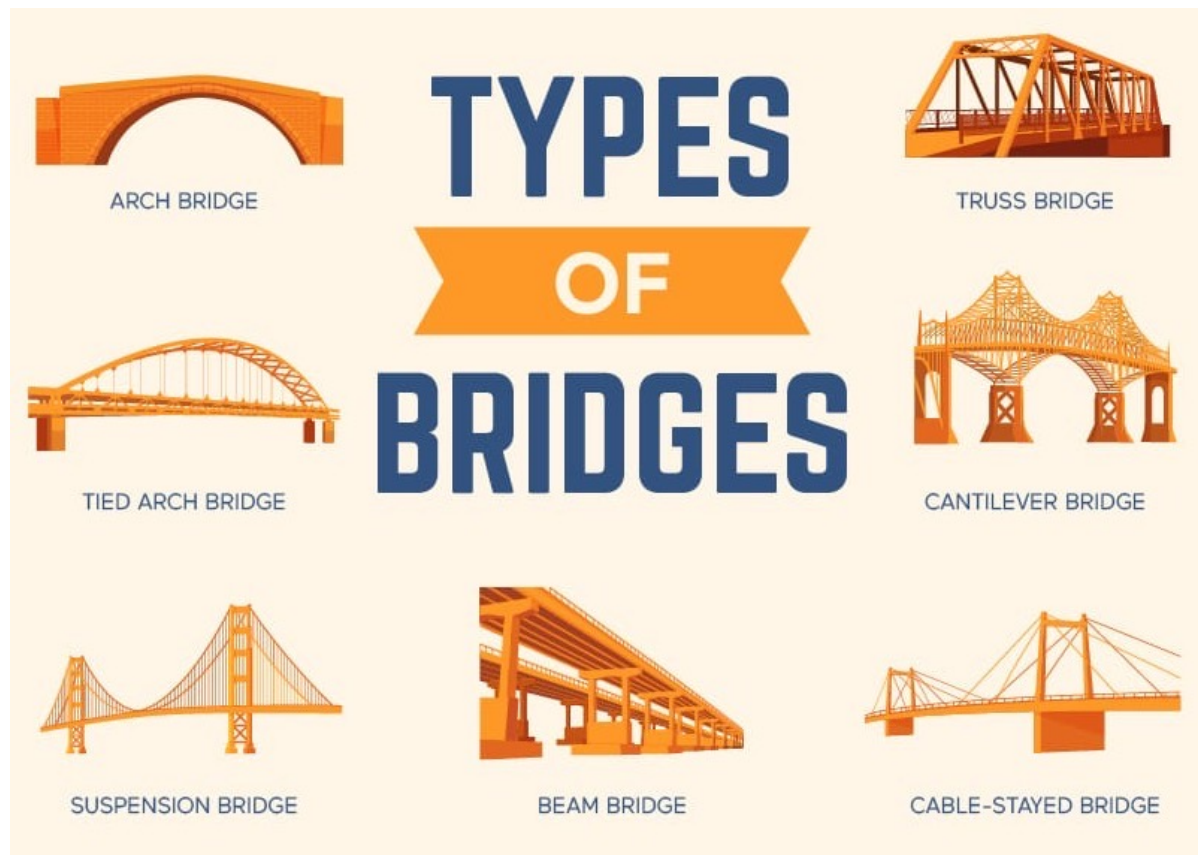
Michael Keeling

El Diseño de Software



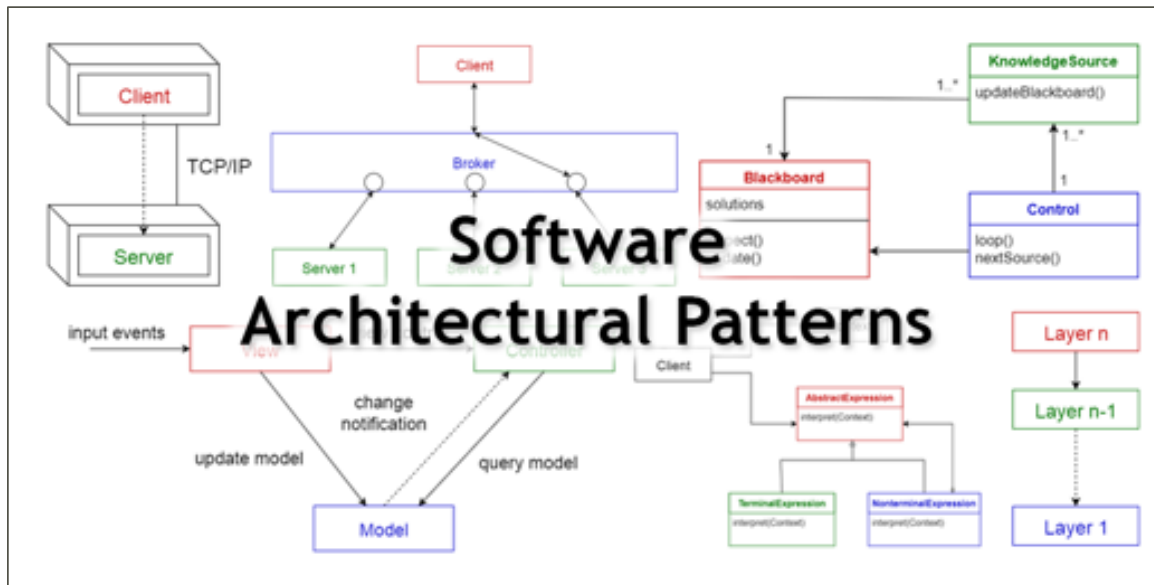
- Planificar antes de construir
- Facilitar la comunicación con múltiples **stakeholders**
- Proveer con un **modelo de análisis** “manejable” para sistemas complejos
- Brindar **prescripciones** para la implementación del sistema
- La arquitectura es un artefacto, pero también parte del proceso de desarrollo (architecting)

Arquitectura, forma y función



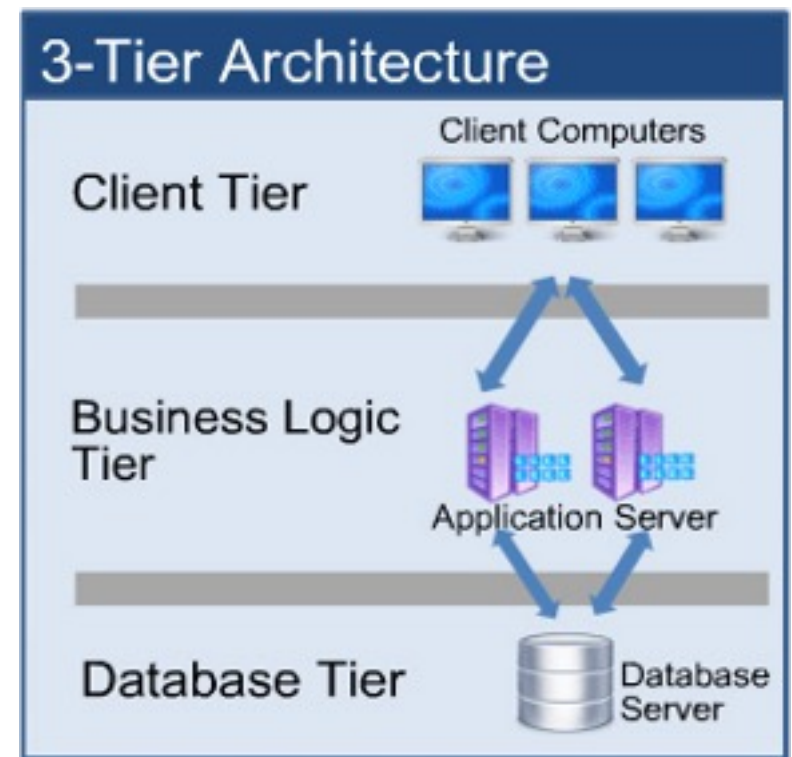
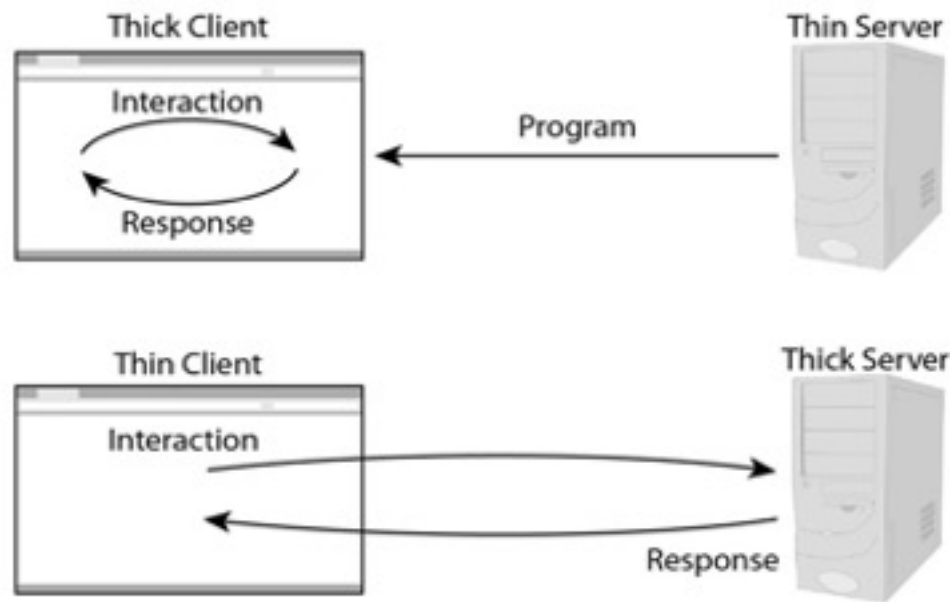
Mary Shaw

Arquitectura, forma y función



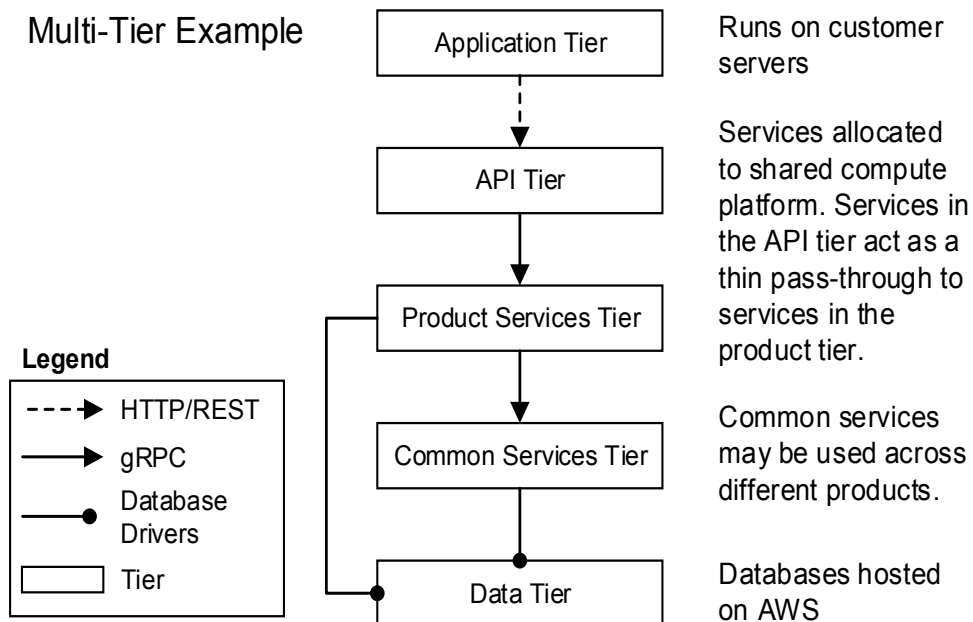
	Layered	Event-driven	Microkernel	Microservices	Space-based
Overall Agility	↓	↑	↑	↑	↑
Deployment	↓	↑	↑	↑	↑
Testability	↑	↓	↑	↑	↓
Performance	↓	↑	↑	↓	↑
Scalability	↓	↑	↓	↑	↑
Development	↑	↓	↓	↑	↓

Ejemplo: Cliente-Servidor



Cliente-Servidor como Patron (estructura)

Multi-Tier Example

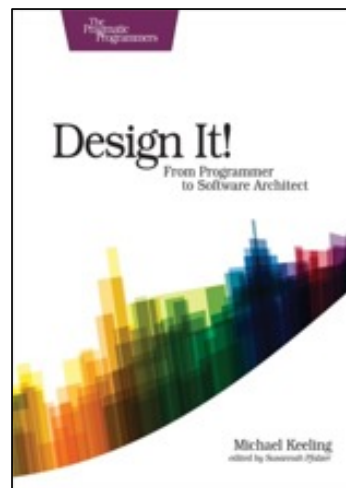
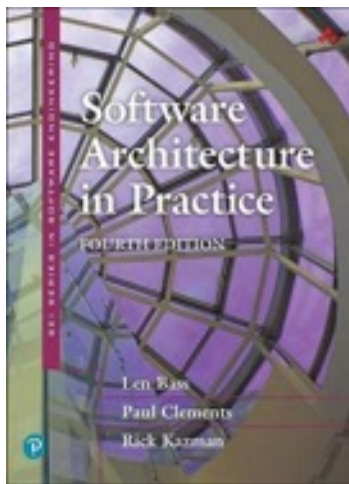


Category	Component & Connector or Allocation
Elements	<i>Tier</i> —a logical grouping of runtime components. There are many ways to partition tiers. Some examples include functional responsibilities, compute platforms, team responsibilities, communication mechanisms, security requirements, and data access.
Relations	<i>Belongs to</i> —used to group components into a tier. <i>Communicates with</i> —shows how tiers or the components within interact with one another. This relation may be specialized to include information about protocols and communication constraints. <i>Allowed to communicate with</i> —indicates which tiers may communicate with components in other tiers. <i>Allocated to</i> —maps tiers to physical compute platforms.
Rules for Use	A component may belong to only one tier. Components within a tier are only allowed to communicate with other components within the same. Additional constraints describing tier communication can enhance reasoning and improve maintainability. A common approach is to allow communication only among adjacent tiers.
Strengths	Promotes security, performance, availability, maintainability, modifiability. Can be used to reason about costs and deployment.
Weaknesses	As a runtime construct, tiers can be difficult to enforce in large systems. Systems with many tiers can inhibit performance and maintainability.

Filosofía



Algunas referencias bibliográficas



Software Engineering Institute
Carnegie Mellon

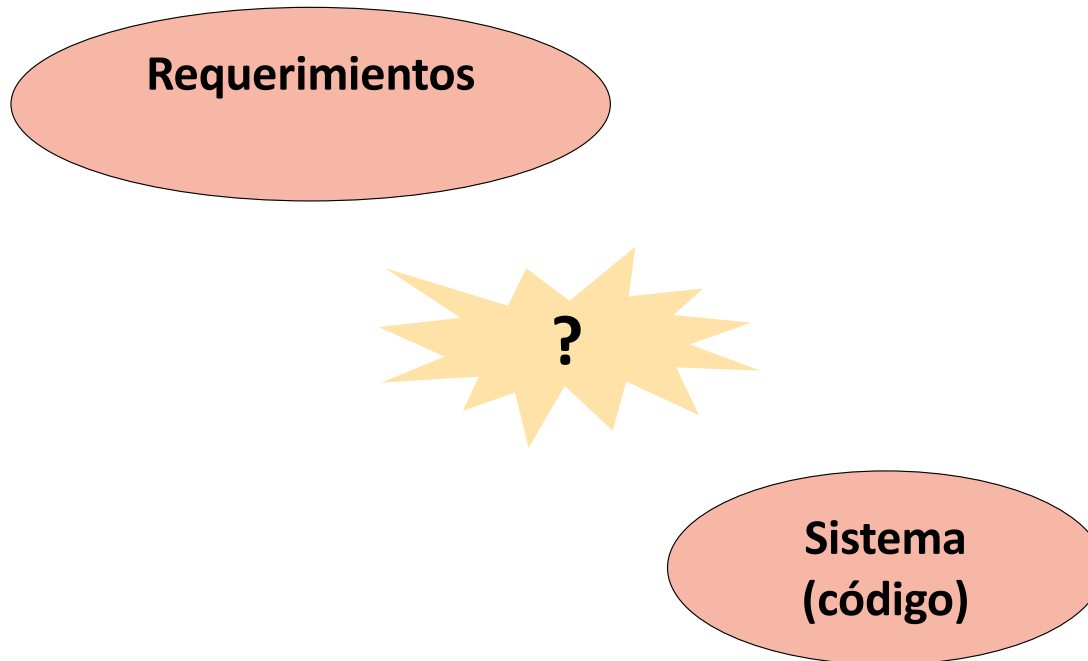


Arquitectura de Software

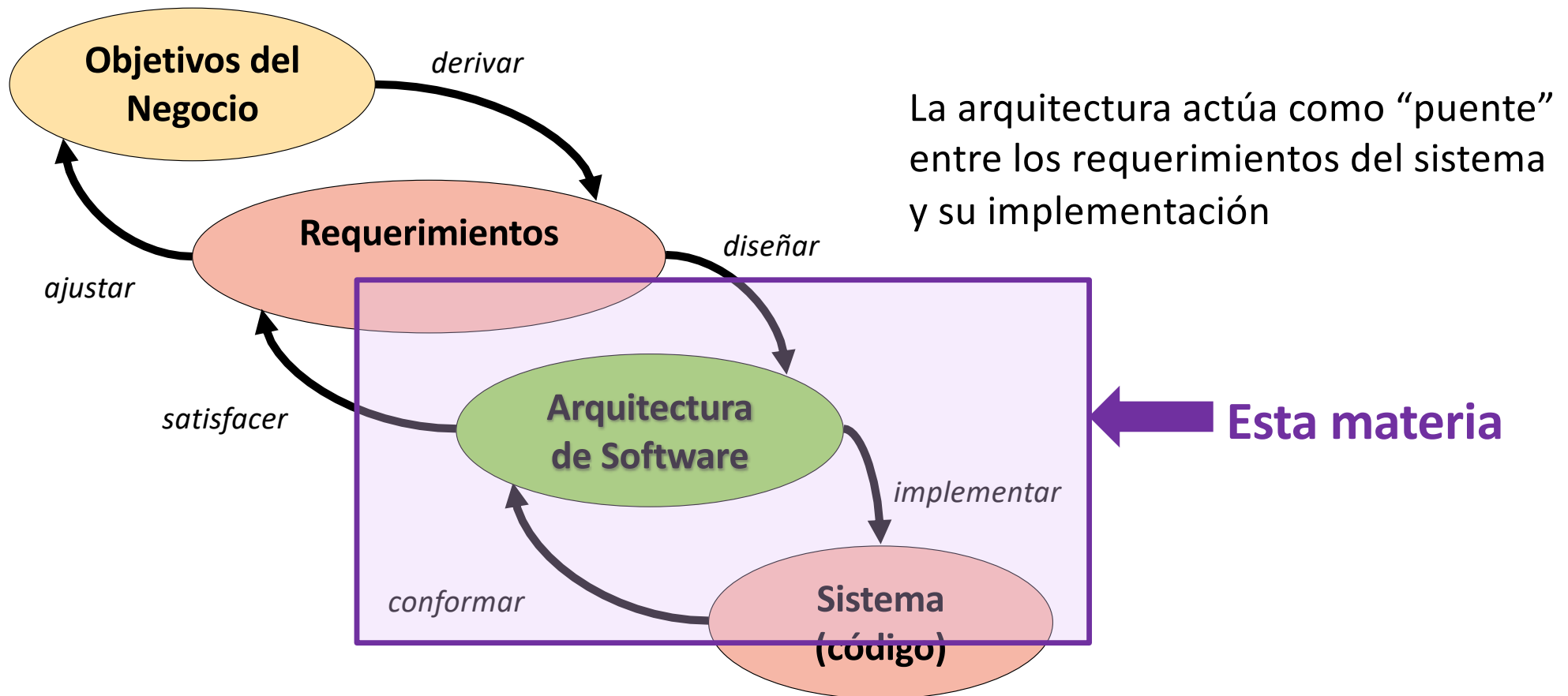
PENSANDO EN DISEÑO (ORIENTADO A OBJETOS, O NO TANTO)

Requerimientos e Implementación

La brecha entre los requerimientos y la implementación es a menudo demasiado grande!

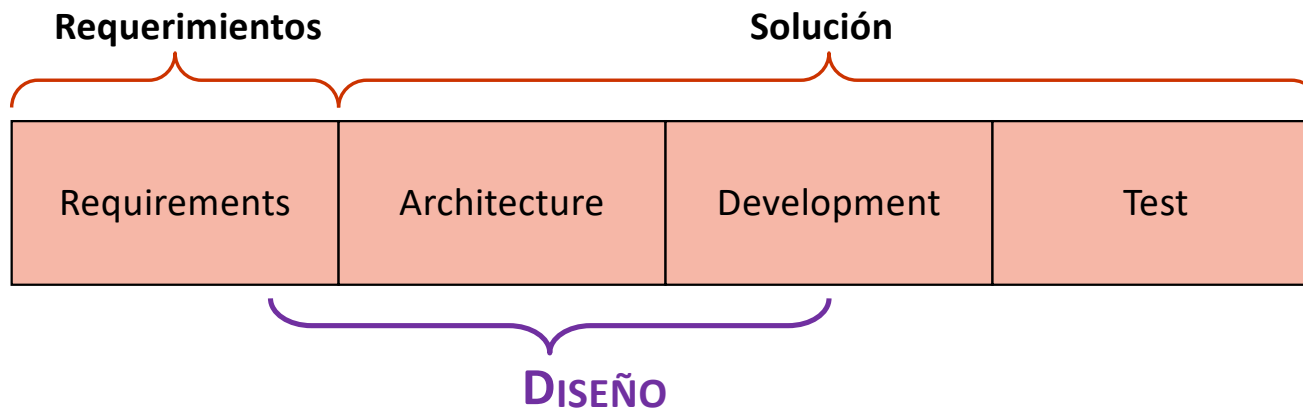


El rol de la Arquitectura

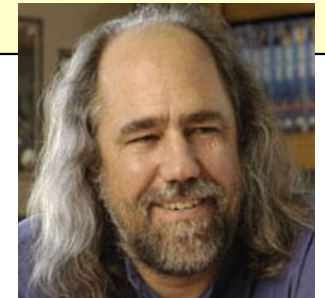


Una definición de arquitectura

"Software architecture is the structure of a software product. This includes elements, the externally visible properties of the elements, and the relationships between the elements." [Bass et al. 2012]

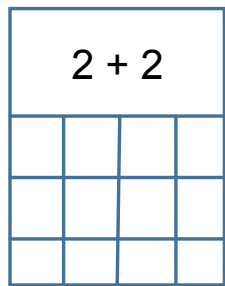


All architecture is design but not all design is architecture. Architecture represents the significant design decisions that shape a system, where significant is measured by cost of change.



Grady Booch

Ejemplo: Mi calculadora App



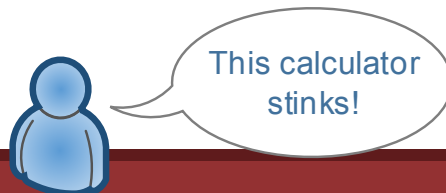
My Amazing!
Calculator App

What if...

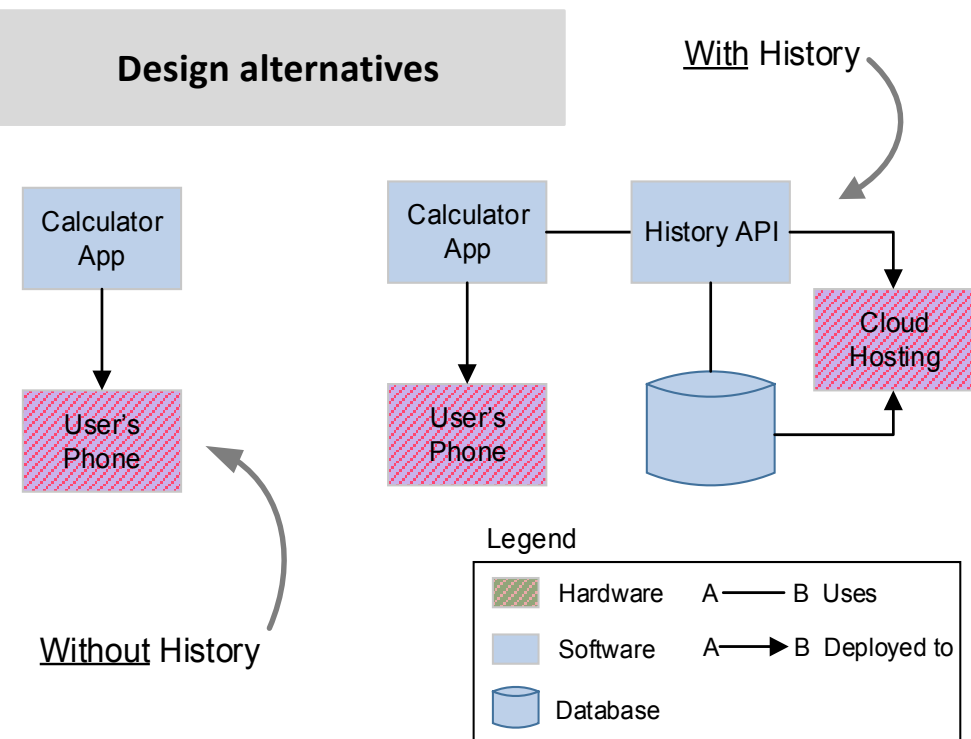
It takes 10 minutes to get an answer?

Sometimes you get the wrong answer?

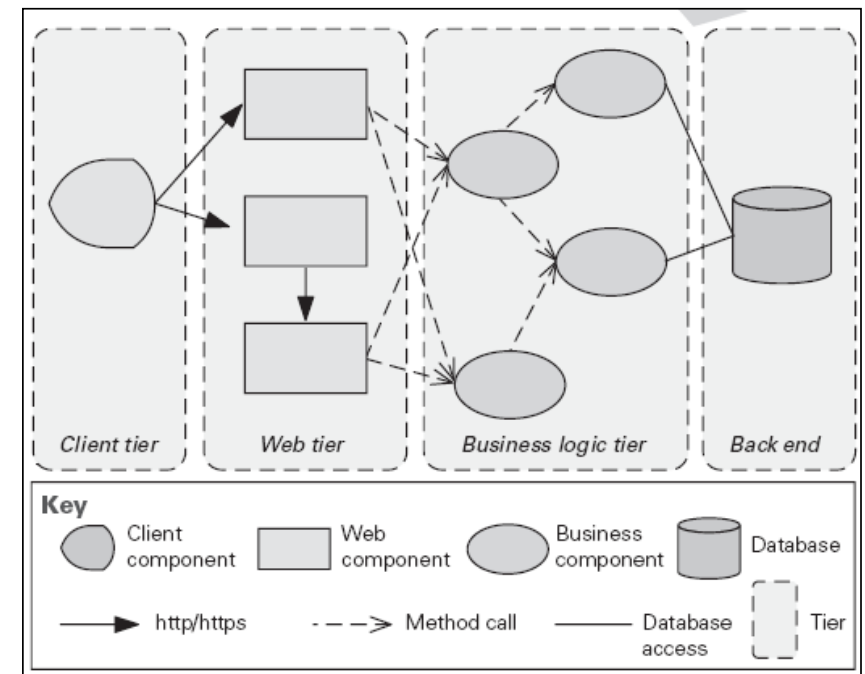
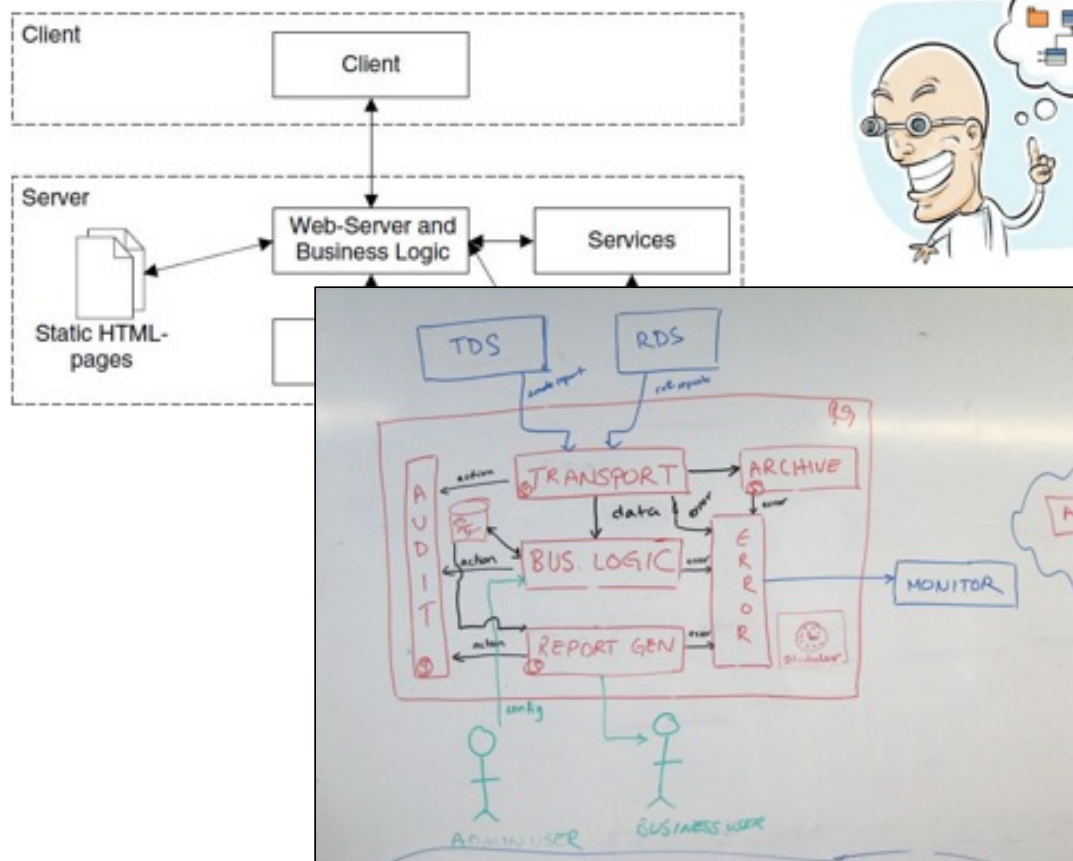
You have to rewrite it to enable subtraction?



Design alternatives



Arquitecturas como “blueprints”



Arquitecturas como decisiones

ADR 1: Use the pipe-and-filter library

This ADR is closely related to [ADR 2](#).

In the past while writing agents, we decided it was a good idea to use [this ADR](#) for context (if it's still around).

We also found that when the engineers who like pipes and filters creep in that violated the pattern. Eventually this defeated the architecturally evident way.

Joe has written a [pipe and filter library](#) that will soon be open source. The architectural style on the project: there are few ways to change things altogether. This stops engineers from accidentally violating the architectural style.

Decision

We will use the pipe-and-filter library to organize all application logic for the training agents.

Status

Accepted

Consequences

All training agents must fit within the context of pipe and filter. For the moment, we think that this is the case. There may be some pipelines that are very small, but will still work.

All filters must be completely decoupled from each other.

Concurrency can be controlled on a per-filter basis. This means we can write faster programs, but we also must be careful to write thread-safe filters if we want to run them concurrently.

Reacting to errors or expected failure modes will be much easier, and can be implemented with the message subscriber feature of the pipe and filter library.

There will be stricter controls on logging and metrics- they must be implemented as message subscribers. This decouples them from the filters entirely, limiting the possibility of mistakes and duplication of logging/metrics code for common error cases, but also means that more work is required to carefully pass good error responses around so that well-formed messages can be constructed and enqueued by the filters.

Uncaught race conditions or errors in the pipeline library may cause failures in production.

“Software architecture = the sum of all important decisions

Important decisions are all decisions that are difficult to change in the course of further development.” [Kruchten 2004]

“

In the context of `<use case/user story u>`,
facing `<concern c>`
we decided for `<option o>`
and neglected `<other options>`,
to achieve `<system qualities/desired consequences>`,
accepting `<downside d/undesired consequences>`,
because `<additional rationales>`.

Ejemplos de Decisiones de Arquitectura

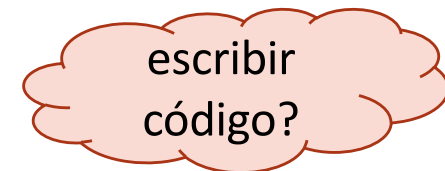
- Uso de un patrón de layers para organizar el sistema
 - Inserción de un intermediario para separar la aplicación que requiere un Web service de la aplicación que provee (e implementa) dicho servicio
 - Utilización de encriptación con clave simétrica?
 - Utilización de un algoritmo que garantiza que no se producen memory leaks?
-
- De qué depende que una decisión sea relevante a nivel arquitectural?



Are you a Software Architect? (Blog)



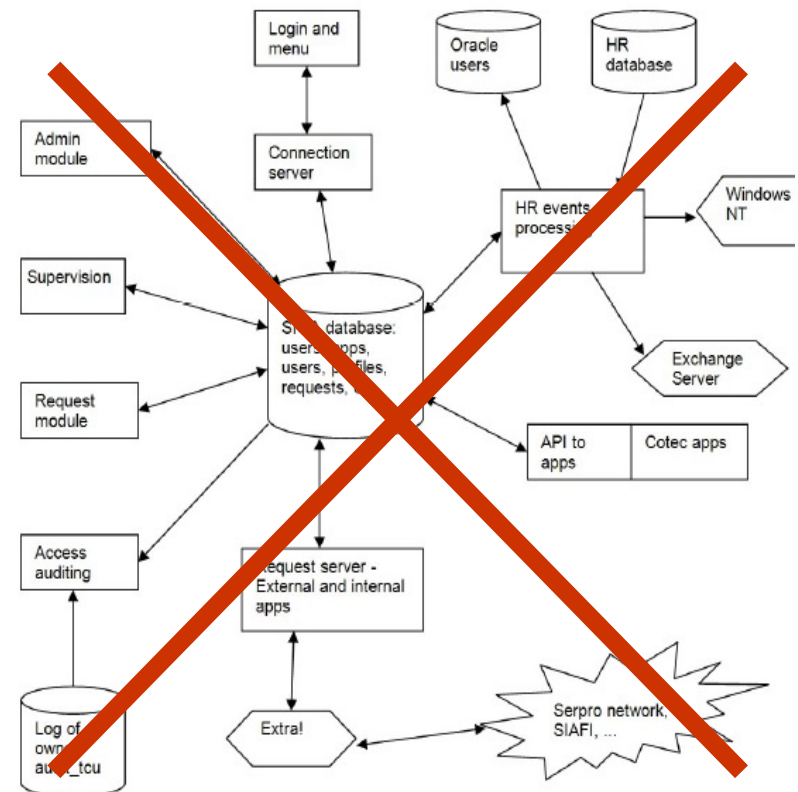
Simon Brown



<https://www.infoq.com/articles/brown-are-you-a-software-architect/>

Concepciones erróneas

- La arquitectura de software NO es la arquitectura del sistema (o de la computadora)
- La arquitectura de software NO es una vista de la funcionalidad
- La arquitectura de software NO es un dibujo “conceptual” de cajas y flechas



Algunos casos de la vida real

- *Un sistema de procesamiento de logs (de películas) para crear un dashboard funciona correctamente, pero presenta problemas cuando se quiere incrementar el número de dispositivos que envían sus logs para procesar.*
- *Un sistema de BI existente debe integrar fuentes de datos no estructuradas (e importantes para el negocio), y el esfuerzo para implementar dicha integración se demora 6 meses más de lo previsto.*
- *Un sistema de reservas turísticas presenta fallas en ciertos servicios (en tiempo de ejecución), que hace que se pierdan las consultas previas a distintos proveedores, y degrada la experiencia de usuario.*



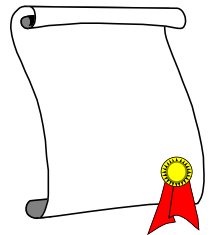
¿Qué tienen en común?: Atributos de Calidad

- **Sin** atributos de calidad
- El sistema implementa la funcionalidad correcta pero:
 - “Anda lento”
 - permite que los hackers roben datos
 - Está caído la mayor parte del tiempo
 - No escala
 - Es difícil realizar cambios o integraciones
 - ...
- **Con** atributos de calidad
- Hablan sobre la calidad esperada del sistema
- Están definidos desde el punto de vista de los stakeholders
- De manera precisa (no ambigua)
- Permite “testear” si el requerimiento de atributo de calidad se satisface o no



Atributos de Calidad (en general)

- **Propiedades** (sistémicas) de un producto de software a través de las cuales los stakeholders juzgan la calidad del producto
 - Performance
 - Seguridad
 - Modificabilidad
 - Disponibilidad
 - Usabilidad
 - Portabilidad
 - Estabilidad
 - Escalabilidad
 - “Webifyability”
 - Sustentabilidad
 - ...



Calidad de la Arquitectura

- **Integridad conceptual**

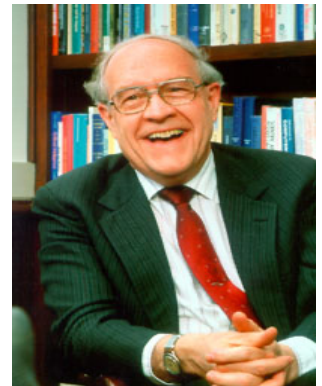
- El sistema se construye a partir de un numero pequeño de estructuras arquitectónicas que interactúan en un numero pequeño de **formas (predeterminadas)**
- Se logra típicamente vía un solo arquitecto, o un grupo de arquitectos pequeño y coordinado

- **Correctitud/completitud**

- La arquitectura debe permitir la satisfacción de todos sus requerimientos funcionales y de calidad

- **Buildability**

- El sistema debe poder construirse con los recursos disponibles



Fred Brooks

Actividad

- Hay que diseñar un sistema Web para el mercado inmobiliario, que permita a compradores y vendedores comprar/alquilar/ofrecer propiedades
 - El sistema usa un modelo de ML para predecir precios de propiedades
- *¿Cuáles son los principales atributos de calidad?*
- *¿Cuáles serían las principales (o mínimas) decisiones de arquitectura para un primer MVP?*



<https://www.infoq.com/articles/minimum-viable-architecture/>



Diseño de Software

IMPLEMENTACIÓN UTILIZANDO OBJETOS

Motivación: un Maestro de Ajedrez

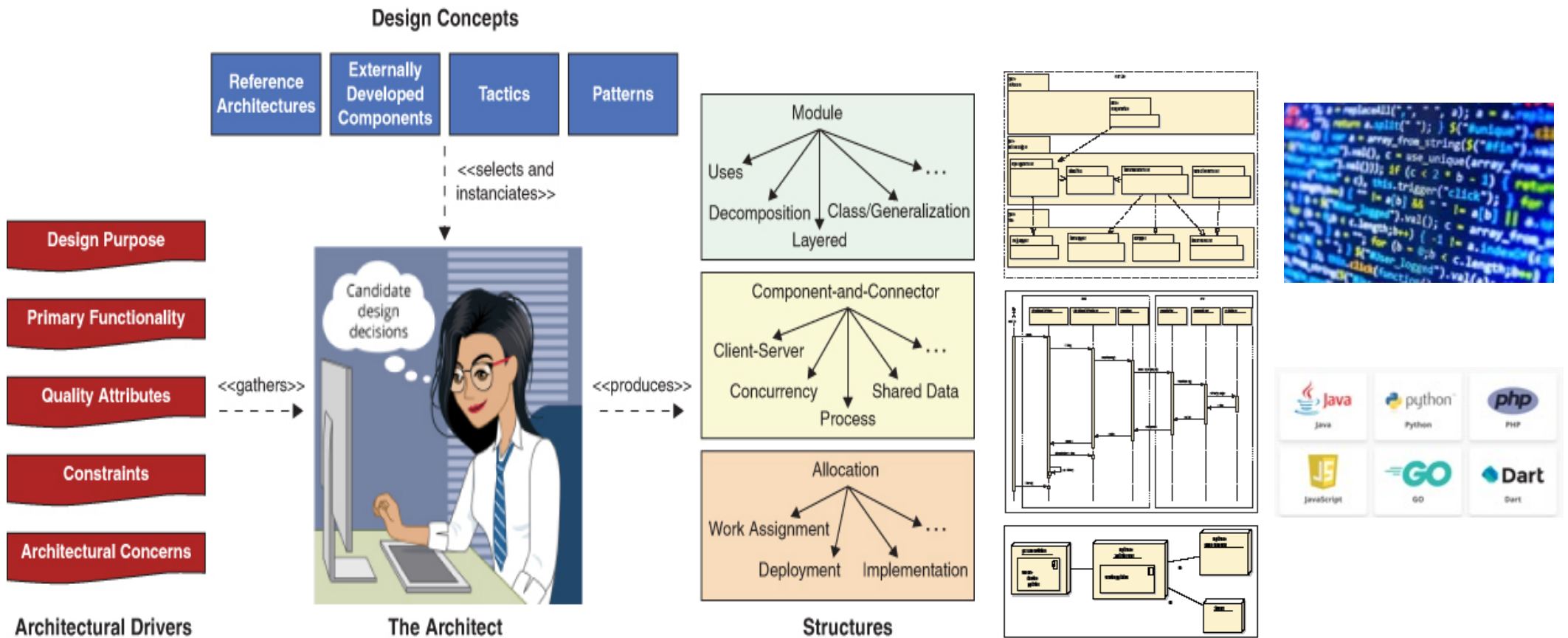
1. Aprender las reglas del juego
2. Aprender los principios
3. Ver jugar a otros
4. Aprender aperturas, teoría de finales, de medio juego, etc.
5. Estudiar las partidas de otros maestros
 - Estas partidas contienen docenas de patrones, que deben ser entendidos, memorizados, y aplicados (con variaciones) a situaciones prácticas



¿Y UN MAESTRO DE DISEÑO/ARQUITECTURA?

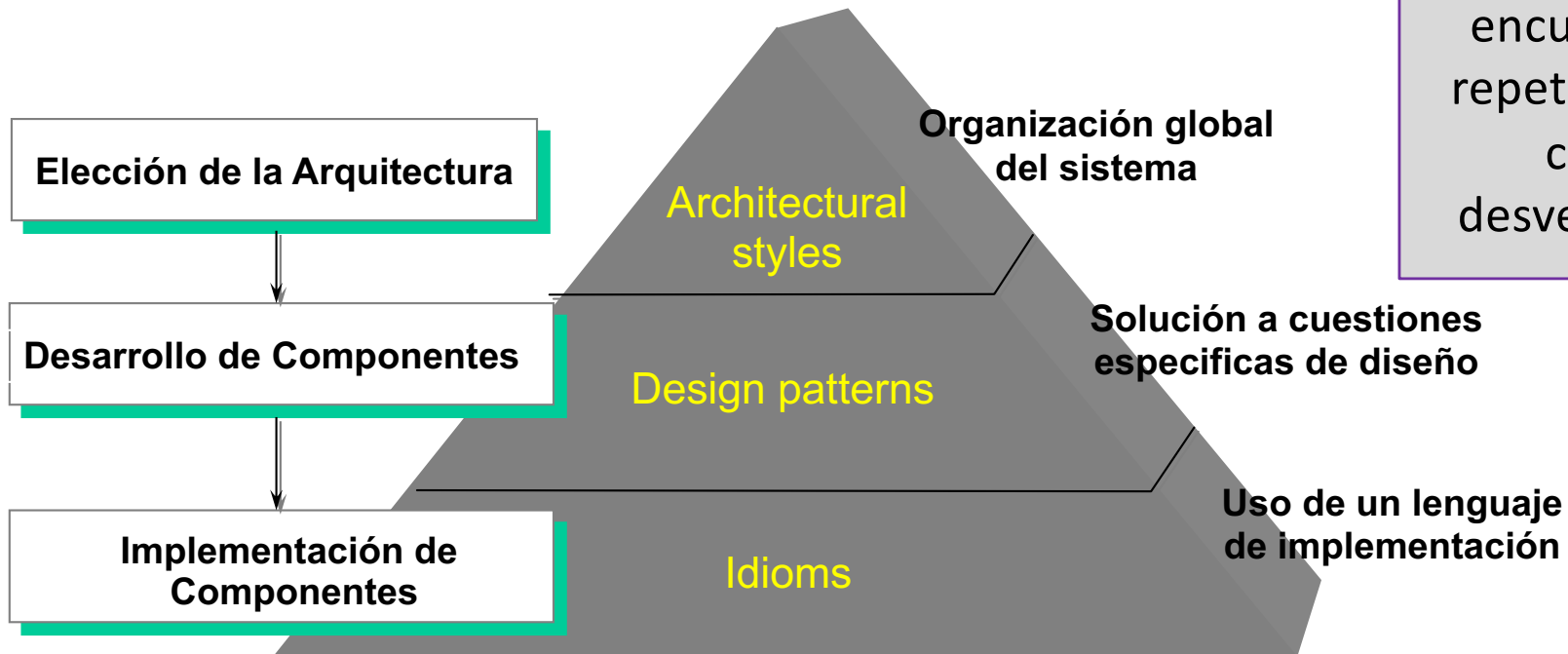
- **Patrones estructurales**
- **Atributos de calidad**
- **Tácticas**
- **Arquitecturas de referencia**
- **Design patterns**

El Proceso de Diseño



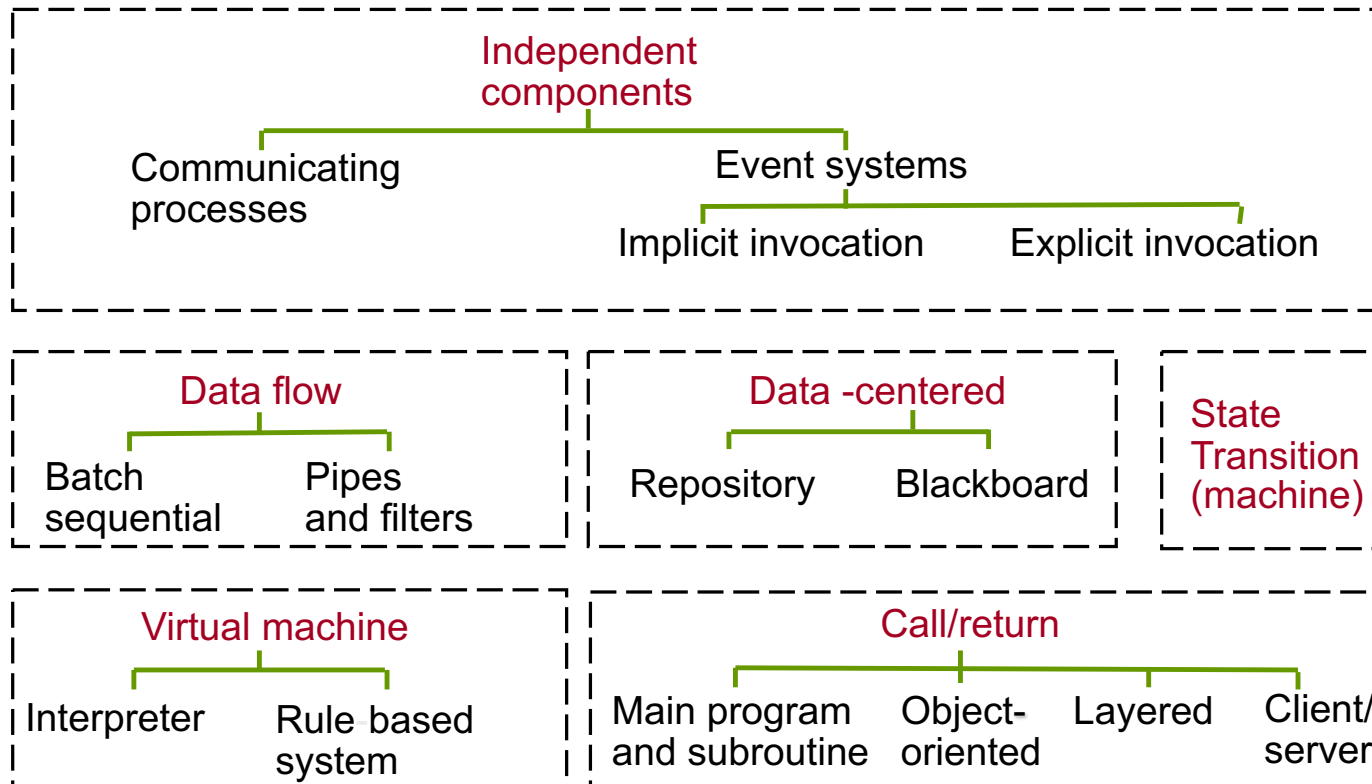
Estilos, patrones de diseño, idioms ...

- Soluciones a distintos niveles de abstracción



Soluciones estructurales (de software) que se encuentran en forma repetida en la práctica, con ventajas y desventajas conocidas

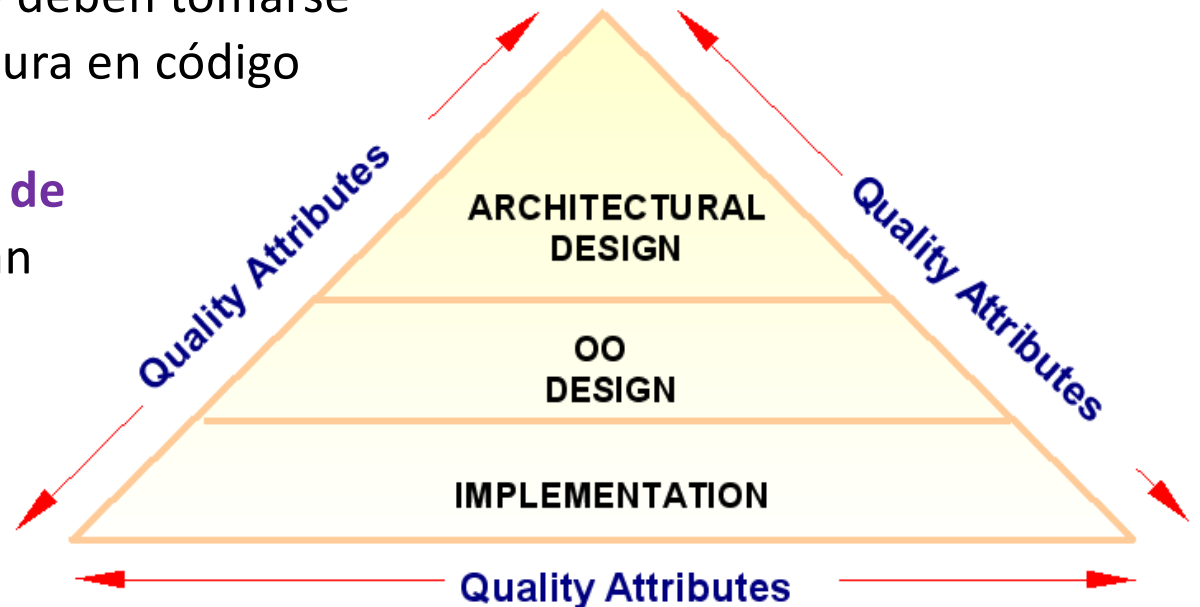
Estilos Arquitectónicos (Taxonomía original)



D. Garlan y M. Shaw

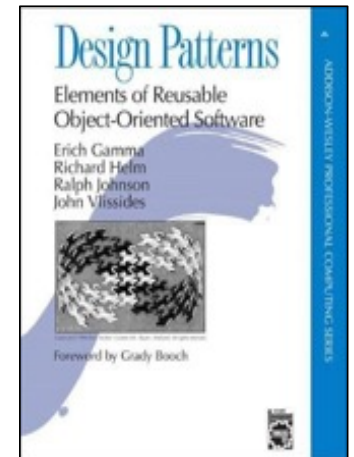
Arquitectura e Implementación

- Las decisiones de arquitectura sirven como **guías** para la implementación de un sistema
- Adicionalmente, existen **otras decisiones (de diseño detallado, de implementación)** que deben tomarse cuando se materializa una arquitectura en código
- Por otro lado, existen **arquitecturas de referencia y tecnologías** que facilitan la implementación



Design Patterns – Gang of four (Repaso)

- Desarrollado mayormente en los '90, aunque la idea luego fue extendida a otros dominios (persistencia, Web, micro-servicios, etc.)
 - Los diseñadores experimentados no utilizan un método de diseño para resolver cada problema específico desde cero
 - Generalmente adaptan soluciones conocidas de problemas similares
- Reconocer y **catalogar estos patrones ofrece un vocabulario de diseño**, a través del cual comunicar soluciones entre diseñadores
- El proceso de diseño deja de ser una actividad intensiva de resolución de problemas, para ser una actividad sistemática de aplicación de conocimiento establecido

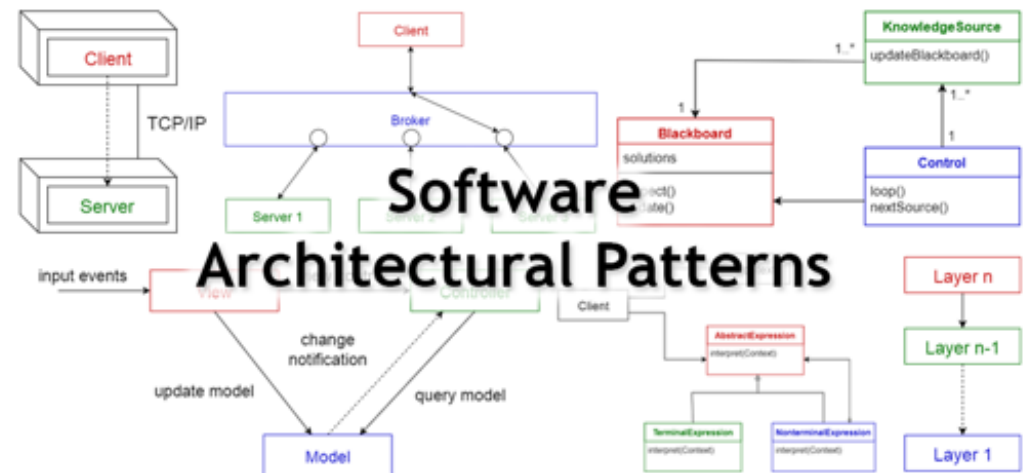


El Catálogo de GoF

JURISDICCIÓN	<u>Creacional</u>	<u>Estructural</u>	<u>Comportamental</u>
Clase	Factory Method	Adapter	Template Method
Objeto	Abstract Factory Prototype Singleton	Adapter Bridge Facade Proxy Flyweight	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor
Compuestos	Builder	Composite Decorator	Iterator Visitor

Diseño con (Design) Patterns

- **Método clásico:** Separar el problema en clases y distribuir responsabilidades entre ellas para resolverlo
- **Mejora de diseño:** Analizar el diseño (actual) de la aplicación y aplicar patrones para aumentar la flexibilidad y reusabilidad de partes específicas del diseño
 - Bajo la forma de refactoring
- **Diseño conducido por patrones:**
Para cada problema relevante: Procurar un patrón que resuelva algún problema similar, y aplicar el patrón al diseño actual



Documentación de un Patrón - 1

NOMBRE DEL PATRÓN

Jurisdicción Caracterización

Intención

¿Qué es lo que el patrón hace? ¿Que problema particular de diseño resuelve?

Motivación

Un escenario en el cual el patrón es aplicable, el problema de diseño que el patrón encara y las estructuras de clases y objetos utilizadas para resolver el problema.

Aplicabilidad

¿Cuáles situaciones el patrón puede ser aplicado? ¿Cuáles ejemplos de diseño inadecuados el patrón puede mejorar? ¿Cómo reconocer tales situaciones?

Participantes

Descripción de las clases y/o objetos participantes en el patrón y sus responsabilidades.

Colaboraciones

Describe cómo los participantes colaboran para realizar sus responsabilidades.

Documentación de un Patrón - 2

Diagrama

Una representación gráfica del patrón utilizando notación UML, extendida con el pseudo-código de los métodos.

Consecuencias

¿Cómo soporta sus objetivos el patrón? ¿Cuáles son los compromisos y resultados de utilizarlo? ¿Qué aspectos de la estructura del sistema permite que sean variados independientemente.

Implementación

¿Cuáles técnicas deberían ser tomadas en cuenta cuando se implementa el patrón?
¿Existen dependencias con aspectos específicos del lenguaje de programación?

Ejemplos

Ejemplos de sistemas reales, en diferentes dominios de aplicación.

Resumen

- Más allá de la escala del sistema, la arquitectura es una **abstracción apropiada** para razonar acerca como se satisfacen ciertos objetivos del negocio o de calidad

<https://www.infoq.com/articles/what-software-architecture/>

- Los **atributos de calidad** tienen una influencia dominante en la arquitectura de un sistema
- La obtención de un diseño satisfactorio depende de la arquitectura (big picture) pero también de su correcta implementación (los detalles)
- El código debe reflejar las **decisiones de diseño arquitectural**



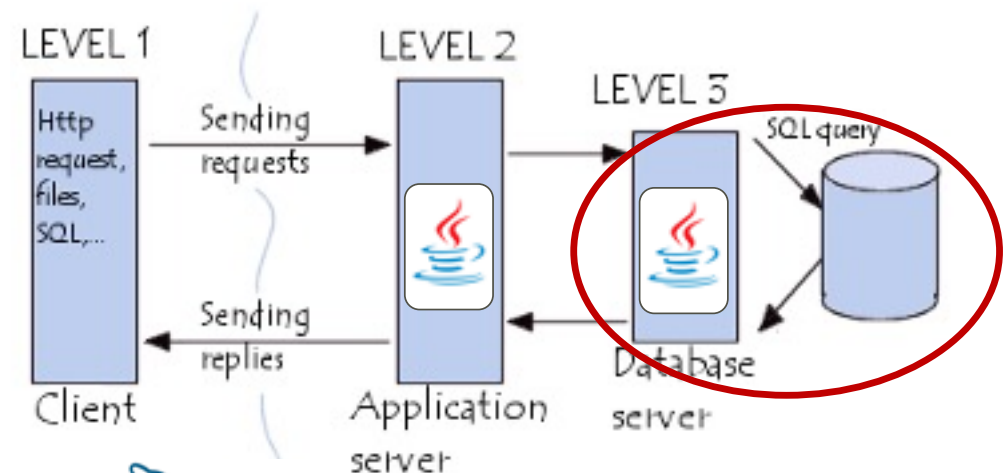
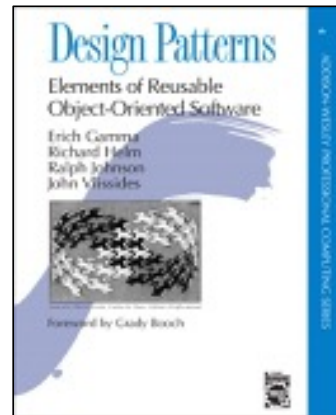
Para la próxima clase (práctica)

1. Volviendo al Cliente-Servidor

- Pensar en un backend implementado en Java
- Agregar un mecanismo de persistencia con base de datos

2. Patrones de diseño

- También en Java



Intercambio



- Andres Diaz Pace
andres.diazpace@isistan.unicen.edu.ar